

단어 정리

-도메인(domain)

▼ 도메인(domain)이란?

도메인 이해력은(소프트웨어/업무 맥락에서) “코드나 기능을 기술적으로만 아는 수준”이 아니라, 그 기능이 속한 ‘업무 세계(도메인)’의 규칙·용어·예외·목표를 정확히 이해하고 설계/개발에 반영하는 능력

서비스/제품이 해결하는 **업무 분야**(예: 이커머스, 결제, 병원 예약, 물류, 인사, 회계 등)

그 분야의 핵심 요소:

용어(유비쿼터스 언어): 주문, 결제, 환불, 정산, 배송완료...

규칙(비즈니스 룰): “환불은 배송완료 후 7일 이내 가능”

예외/케이스: “부분 환불”, “쿠폰 동시 적용 불가”, “분할 배송”

목표/지표: 전환율, 이탈률, 정산 정확도, 리드타임 등

▼ 도메인 이해력이 높은 사람의 특징

용어를 정확히 쓰고, 사용자/기획/운영과 같은 단어를 공유함

“이 기능은 이렇게 만들면 된다”보다

“왜 필요한가 / 어떤 규칙 때문에 이런 설계가 필요한가”를 설명 가능

옛지 케이스를 미리 질문함

예: “취소는 결제 전/후가 다르죠?”, “부분취소 가능한가요?”

요구사항을 코드로 옮길 때 **모호함을 줄이고 규칙을 명시화**함

변경(정책 변경, 법규 변경)에 강한 구조로 설계하려고 함

▼ 도메인 이해력이 부족할 때 흔히 생기는 문제

화면/기능은 되는데 **정산/권한/상태 전이** 같은 핵심 룰이 틀림

“주문 상태” 같은 중요한 개념이 코드 곳곳에서 제각각 정의됨

예외 케이스 대응이 늦어져 운영 이슈/CS 폭증

▼ (개발 관점) 도메인 이해력을 키우는 방법

도메인 이벤트/상태 흐름을 그려보기 (주문 생성 → 결제 → 배송 → 완료 → 환불)

“정의서 만들기”

핵심 용어(Glossary)

상태(State)와 전이(Transition) 규칙

불변조건(Invariant): “결제완료 없이 배송 시작 불가”

실제 사례/CS/운영 시나리오로 테스트 케이스 만들기

기획자/운영자에게 질문할 때 템플릿:

“언제/누가/무엇을/어떤 조건에서/예외는?”

가능하면 DDD 관점으로:

엔티티/값객체/애그리게잇, 바운디드 컨텍스트 분리 등

▼ 도메인 이해력

도메인 이해력은 기능을 기술적으로 구현하는 것을 넘어, **업무 용어·규칙·예외 케이스를 정확히 이해하고 설계에 반영하는 능력**입니다.

예를 들어 “환불” 기능을 만들 때 ‘결제 완료 후에만 환불 가능’, ‘부분 환불 가능 여부’, ‘배송 완료 후 7일 제한’ 같은 규칙을 먼저 확인하고 로직/상태를 설계하는 게 도메인 이해력입니다.

- CI·CD / DevOps / Cloud / On-premise

CI/CD:

코드 변경이 생길 때마다 자동으로 빌드, 테스트, 배포 과정을 수행해 품질과 배포 속도를 높이는 개발 방식

▼ CI (Continuous Integration, 지속적 통합)

의미: 개발자가 코드를 자주(작게) 메인 브랜치에 합치고, 그때마다 **자동 검증**을 돌려 “통합 문제를 빨리 발견”하는 방식.

CI가 자동으로 하는 대표 작업

코드 체크아웃 → 의존성 설치

빌드(컴파일)

테스트(유닛 테스트 중심)

정적분석(lint), 포맷 검사

보안/취약점 스캔(옵션)

결과 리포트(성공/실패) + PR에 체크 표시

핵심 포인트

“합치기 전에(또는 합치자마자) 깨지는 걸 빨리 잡자”

작은 변경을 자주 통합할수록 충돌/버그 비용이 줄어듦

▼ CD (Continuous Delivery / Continuous Deployment)

CD는 “**배포까지의 흐름을 자동화**”하는 개념인데, 마지막 배포 단계가 자동이나 아니냐로 2가지로 나뉘어요.

A. Continuous Delivery (지속적 전달)

의미: 언제든지 배포 가능한 상태를 자동으로 만들어두되, **실제 프로덕션 배포는 사람의 승인/버튼으로 진행.**

예: 스테이징까지 자동 배포 + 테스트 완료

→ “승인하면” 프로덕션 배포

B. Continuous Deployment (지속적 배포)

의미: 테스트/검증이 통과하면 **프로덕션까지 자동 배포.**

예: main에 머지되면 자동으로 프로덕션 배포까지 쭉 진행

정리

Delivery = “배포 준비 자동, 배포는 승인”

Deployment = “배포까지 자동”

▼ CI/CD 파이프라인(흐름) 예시

개발자가 PR/커밋 푸시

CI 단계

빌드 + 테스트 + 품질검사

산출물 생성(artifact)

예: Docker 이미지, 빌드된 파일(zip), 패키지

스테이징 배포

통합/E2E 테스트(필요 시)

프로덕션 배포

Delivery면 승인 후 배포

Deployment면 자동 배포

모니터링/알림 + 필요 시 롤백

▼ CI/CD에서 자주 나오는 용어

Pipeline: 자동화된 전체 흐름(단계들의 묶음)

Stage/Job/Step: 파이프라인을 구성하는 단계/작업 단위

Artifact: 빌드 결과물(배포에 쓰는 파일/이미지)

Runner/Agent: 파이프라인을 실제로 실행하는 머신/환경

Environment: dev / staging / production 같은 배포 대상 구분

Rollback: 배포 실패 시 이전 버전으로 되돌리기

Blue-Green / Canary 배포: 위험을 줄이기 위한 배포 전략

▼ 왜 쓰는가(효과)

반복 작업(빌드/테스트/배포) 자동화 → **실수 감소**

문제를 빨리 발견 → **수정 비용 감소**

배포가 일정한 절차로 실행 → **예측 가능/재현 가능**

▼ DevOps

개발(Dev)과 운영(Ops)을 한 흐름으로 묶어서 **빠르고 안정적으로 배포/운영**하는 문화·프로세스·자동화(= CI/CD, IaC, 모니터링 포함).

▼ Cloud:

서버/DB/스토리지 같은 자원을 **필요할 때 빌려 쓰는 방식**. 확장과 자동화가 쉬워 DevOps와 궁합이 좋음.

▼ On-premise(온프레미스):

회사가 **직접 보유한 서버/데이터센터**에서 운영. 통제/보안 장점이 있지만, 인프라 운영 책임이 커서 자동화(IaC, K8s 등)가 중요.

-WMS (창고관리시스템)

WMS는 창고에서 일어나는 입고-적치-재고-피킹-패킹-출고-반품을 관리해서 재고 정확도, 처리속도, 출고 정확도를 올리는 시스템.

보통 ERP/OMS/TMS/택배사 API 등과 연동해서 "현장 실행"을 관리함.

-AI / ML / DL / Generative AI

▼ AI: 지능적 문제 해결 전체(규칙 기반도 포함)

AI (Artificial Intelligence, 인공지능)

가장 큰 상위 개념

"사람처럼 판단/추론/문제 해결을 하게 만드는 기술" 전반

예: 규칙 기반 전문가 시스템(옛날 방식), 최적화/탐색, 머신러닝 등 모두 포함

▼ ML: 데이터로 예측/분류/추천을 "학습"

ML (Machine Learning, 머신러닝)

AI의 한 분야

사람이 규칙을 일일이 코딩하기보다, **데이터로부터 패턴을 학습**해서 예측/분류/추천을 수행

대표 예:

스팸 메일 분류(분류)

집값 예측(회귀)

고객 군집화(클러스터링)

상품 추천(추천/랭킹)

▼ DL: 신경망 기반 ML(이미지/음성/텍스트에 강함)

DL (Deep Learning, 딥러닝)

ML의 한 분야

“신경망(Neural Network)”을 여러 층(Deep)으로 쌓아 **복잡한 패턴을 잘 학습**하는 방법

강점: 이미지/음성/자연어처럼 **비정형 데이터**에서 성능이 좋음

대표 모델/구조:

CNN(이미지)

RNN/LSTM(시계열/텍스트, 과거에 많이 사용)

Transformer(현재 NLP/생성형의 핵심)

▼ **생성형 AI**: 텍스트/이미지/코드처럼 **새 콘텐츠를 생성**

생성형 AI(Generative AI)

“새로운 콘텐츠를 만들어내는” AI

대부분의 최신 생성형 AI는 **딥러닝(특히 Transformer)** 기반

만들어내는 것:

텍스트(챗봇, 요약, 번역)

이미지/영상

음성/음악

코드

핵심 특징:

분류/예측이 아니라 **생성(Generation)** 이 목적

“다음 토큰(단어/픽셀 등)을 확률적으로 예측”하는 방식이 흔함

포함 관계

AI ⊃ ML ⊃ DL

Generative AI는 “기능/목적” 관점의 분류라서,

전통적으로도 생성 모델이 있었지만(예: GAN, VAE),

요즘 말하는 생성형 AI는 **대부분 DL 기반**이고 특히 ****LLM(대규모 언어모델)****이 중심이에요.

자주 헷갈리는 포인트

“AI = 다 딥러닝”은 아님 (AI에는 규칙 기반/탐색/최적화도 포함)

“Generative AI = ChatGPT만”도 아님 (이미지/음성/영상/코드 생성 포함)

생성형 AI는 보통 **학습(Training)**에 데이터가 많이 필요하고, **추론(Inference)** 때는 프롬프트로 결과를 생성

AI Service: 모델 자체가 아니라, 모델을 ****API/화면으로** 제공하고 운영(배포, 모니터링, 보안, 비용)**까지 포함한 “서비스 형태”.

-LLM / sLLM / 멀티모달 AI / 에이전틱 AI

▼ **LLM:** 대규모 텍스트를 학습한 언어 모델로 **요약/번역/질의응답/문서·코드 생성**을 수행.

LLM (Large Language Model)

정의: 대규모 텍스트 데이터로 학습해서 “다음 토큰(단어 조각)”을 예측하는 방식으로 **텍스트를 이해·생성**하는 모델.

하는 일: 질의응답, 요약, 번역, 글쓰기, 코드 작성, 분류 등

핵심 특징

자연어 인터페이스가 강력함(사람 말로 지시 가능)

다만 “정답을 조회”한다기보다 **그럴듯한 생성**을 하므로, 틀릴 수 있음(환각)

예시(개념): GPT 계열, Llama 계열 등

고객이 “환불 규정 알려줘”라고 물으면, LLM이 내부 정책 문서를 바탕으로 핵심만 요약해서 자연스럽게 답변하게 만들 수 있습니다.

요약: 언어를 다루는 모델(두뇌)

▼ **sLLM(Small LLM)** 개념

LLM을 ‘더 작게’ 만든 모델: 파라미터 수/모델 크기/연산량을 줄여서 가볍게 만든 언어모델

목표: “최고 성능”보다는 **빠르고 싸고, 기기/서버 제약에서 잘 도는 것**

왜 쓰나(장점)

비용↓: API/서버 GPU 비용 감소

지연시간↓: 응답이 빠름(실시간 서비스에 유리)

온디바이스 가능성↑: 노트북/모바일/엣지 기기에서도 구동(프라이버시에도 유리)

특정 업무에 충분: 범용 대화는 약해도, “정해진 업무”에서는 성능이 충분한 경우가 많음

한계(주의)

범용 추론/복잡한 문제 해결은 대형 LLM보다 약한 편

긴 문맥 처리, 고난도 코딩/수학, 미묘한 지시 따르기에서 성능 차이가 날 수 있음

그래서 보통 **RAG(검색 기반) + 톨 사용 + 가드레일**로 보완하거나,
작업을 나눠 중요한 단계만 큰 모델로 보내는 "혼합 전략"을 씁니다.

sLLM이 잘 맞는 사용 사례

고객센터/FAQ처럼 범위가 좁고 답변 근거가 정해진 업무

사내 문서 검색 요약(RAG) + 짧은 답변 생성

분류/라벨링(메일 분류, 티켓 분류, 의도 분류)

개인정보 때문에 외부 전송이 어려운 온프레미스/온디바이스 환경

▼ LLM vs sLLM 차이점

1) 핵심 차이점 (무엇이 달라지나)

모델 크기/연산량

LLM: 큼 → 연산량·메모리 요구량 큼

sLLM: 작음 → 가볍고 빠름

성능 범위(범용성)

LLM: 다양한 주제/난이도에서 평균적으로 강함(범용)

sLLM: 특정 범위/업무에서는 충분하지만, 고난도/장문/복잡 추론에서 약해질 수 있음

비용과 운영 난이도

LLM: 추론 비용(토큰당 비용), GPU 비용, 지연시간 부담이 커지기 쉬움

sLLM: 비용↓, 지연시간↓, 운영/확장/온프레미스 배포가 쉬운 편

배포 형태

LLM: 보통 클라우드/API로 쓰는 경우가 많음(온디바이스는 어려운 경우 많음)

sLLM: 서버 내 자체 호스팅, 엣지/온디바이스까지 선택지가 넓어짐

2) LLM의 강점

복잡한 추론/문제 해결(다단계 reasoning, 어려운 코드/수학/기획)

긴 문맥 이해와 지시사항 준수(모델에 따라 차이 있지만 대체로 유리)

범용성: 도메인이 넓거나 질문 유형이 예측 불가능한 서비스에 유리

생성 품질: 문장 자연스러움, 스타일, 요약 품질이 평균적으로 좋음

LLM의 단점

비용이 큼: 트래픽 많으면 비용이 급증

지연시간 증가: 응답이 느려질 수 있음

배포 제약: 온프레미스/온디바이스가 어렵거나 고비용

(공통 단점) **환각/오답 가능성:** 모델이 커도 “그럴듯한데 틀린 말”을 할 수 있어 검증이 필요

3) sLLM의 강점

빠름(낮은 latency): 실시간 응답이 중요한 서비스에 유리

저렴함: 대량 호출/대규모 사용자 서비스에서 특히 강점

배포 유연성: 사내망/온프레미스/엣지/온디바이스까지 가능성이 큼

특정 업무에 최적화 가능: 범위가 좁은 업무(분류, 정형 응답, 내부 문서 기반 Q&A 등)는 sLLM로도 충분한 경우가 많음

sLLM의 단점

고난도 추론·복잡한 지시에서 성능이 떨어질 수 있음

긴 문서/긴 대화 맥락 처리에서 불리할 수 있음

“아는 척”이 더 두드러져 보일 수 있어(모델 용량 한계) **RAG/가드레일/후처리**가 더 중요해지는 경우가 많음

4) 언제 무엇을 쓰면 좋은가 (실무 선택 가이드)

LLM이 유리한 경우

사용자가 뭘 물어볼지 예측이 어렵고 범위가 넓음

고난도 reasoning/코딩/기획/복잡한 요약이 핵심 가치

sLLM이 유리한 경우

트래픽이 많고 비용/속도가 가장 중요

업무 범위가 비교적 좁고(FAQ/분류/서식화) 정답 근거를 시스템이 제공할 수 있음

사내망/개인정보 때문에 온프레미스·온디바이스가 필요

현업에서 흔한 베스트 조합

“대부분은 sLLM” + “어려운 요청만 LLM로 라우팅” (하이브리드)

또는 sLLM에 **RAG(문서검색)**을 붙여 정확도를 끌어올리는 방식

▼ **멀티모달 AI**: 텍스트뿐 아니라 **이미지/음성/영상** 등 여러 입력을 함께 이해/생성.

Multimodal AI (멀티모달 AI) 사용자 표현 “multimodel”은 보통 “multimodal”을 뜻하는 경우가 많아요

정의: 텍스트뿐 아니라 **이미지, 음성, 영상** 등 여러 형태(modality)의 입력/출력을 함께 다루는 AI.

예시 기능

이미지 보고 설명하기(시각 이해)

문서/스크린샷에서 정보 추출

음성 받아쓰기(STT), 음성 합성(TTS)

텍스트 → 이미지 생성, 텍스트 → 영상 생성 등

관계:

LLM은 기본적으로 텍스트 중심

멀티모달은 **언어 모델 + 비전/오디오 모델을 결합**하거나, 하나의 모델이 여러 모달을 함께 처리하도록 만든 형태

에이전틱(Agentic) AI: 답만 생성하는 게 아니라 **목표를 받고 계획 → 도구 사용 → 여러 단계 실행**으로 업무를 끝까지 수행.

sLLM: 보통 **Small LLM** 의미. 가볍고 빠르고 온프레/저비용에 유리하지만, 복잡한 추론은 약해 **RAG/튜닝**으로 보완.

예시: 에러 화면 스크린샷을 올리면, 멀티모달 AI가 화면의 메시지를 읽고 “어떤 설정을 확인해야 하는지”를 텍스트로 안내해줄 수 있습니다.

요약: 언어 + 이미지 / 음성 등까지 다루는 모델(감각 확장)

▼ **Agentic AI (에이전틱 AI, 에이전트형 AI)**

정의: 단순히 답변만 하는 게 아니라, **목표를 받고 스스로 계획을 세워 여러 단계를 실행**하는 AI(필요하면 도구/앱도 사용).

LLM이 “두뇌(추론/언어)” 역할이라면, Agentic AI는 거기에:

계획(Plan)

행동(Act): 검색, DB 조회, 코드 실행, 이메일 작성, 캘린더 등록 등 “툴 사용”

반성/검증(Reflect/Evaluate): 결과가 맞는지 체크하고 재시도

메모리/상태 관리: 이전 맥락, 작업 상태를 유지
를 붙인 형태에 가까워요.

예시(상황)

“회의록 정리해서 액션아이템 뽑고, 담당자에게 메시지 보내고, 캘린더에 일정 잡아줘”

“오류 로그 분석 → 원인 후보 정리 → 재현 단계 만들기 → 수정 PR 초안 작성”

장점: “업무를 끝까지” 진행 가능

주의점: 권한/안전(무엇을 자동으로 실행할지), 검증(잘못된 행동) 설계가 중요

요약: 모델(LLM / 멀티모달)를 사용해 목표 달성까지 스스로 계획, 도구 실행하는 시스템(행동하는 형태)

-AI Service

AI Service는 보통 **AI(특히 ML/LLM)**를 제품 기능으로 제공하는 서비스/시스템을 뜻해요.
“모델 자체”보다 **사용자가 쓰는 형태로 운영되는 엔드투엔드**를 말합니다.

▼ AI Service의 정의

데이터/모델/인프라를 묶어 예측·추천·생성 같은 기능을 API나 앱 기능으로 안정적으로 제공하는 서비스

예를 들어 “쇼핑몰 상품 추천” 서비스는 사용자 클릭/구매 로그를 모아서 추천 모델을 학습하고, 앱에서 실시간 추천을 제공하면서 성능(A/B 테스트)과 장애/지연시간을 계속 모니터링하는 형태입니다.

▼ 구성 요소(필수 블록)

입력(Inputs)

사용자 텍스트, 이미지, 음성, 클릭로그, 문서 등

AI 코어

ML 모델(분류/예측/추천) 또는 LLM/멀티모달 모델(생성/요약/QA)

(LLM이면) 프롬프트, 시스템 지침, 톨 호출, 가드레일 등 포함

데이터 계층

학습 데이터(오프라인), 운영 데이터(로그)

RAG용 문서 저장소 + 벡터DB(필요 시)

서빙/인프라

모델 서빙 서버, GPU/CPU, 캐시, 스케일링, 모니터링

품질/안전 장치

정확도 평가, A/B 테스트, 안전 필터링, 개인정보 마스킹, 정책 준수

피드백 루프

사용자 피드백 수집 → 재학습/프롬프트 개선/룰 개선

▼ AI Service 유형(대표 3가지)

Prediction형(전통 ML): 스팸 분류, 이탈 예측, 수요 예측

Recommendation/Ranking형: 상품 추천, 콘텐츠 피드 랭킹

Generative형(LLM 중심): 챗봇, 요약, 문서 QA, 코드/콘텐츠 생성

▼ 장점 / 단점(서비스 관점)

장점

자동화/개인화로 사용자 경험 개선

운영 효율(상담/문서 처리 등) 증가

데이터가 쌓일수록 개선 여지 큼

단점/리스크

오답/환각(특히 생성형) → 검증/근거제시 필요

비용/지연시간(LLM, GPU) 관리가 중요

개인정보/저작권/보안 이슈 설계가 필요

모델 업데이트/데이터 드리프트로 성능이 변함 → 지속 모니터링 필요

▼ 면접에서 자주 쓰는 "AI Service 설계 키워드"

SLA/Latency/Cost(응답속도·비용·가용성)

Observability(로그/모니터링/트레이싱)

Evaluation(오프라인 지표 + 온라인 A/B)

Safety & Compliance(PII, 정책, 필터링)

RAG/Cache/Fallback(정확도·비용·안정성 확보)

2026-08-26

2026-08-27

2026-08-28